

Implementation of Web Application Firewall and Attack Simulation on DVWA



Ravintharan Sanjiv
Aspiring Cybersecurity Analyst
SafeLine Web Application Firewall
May 2026

All attacks were conducted within a controlled laboratory environment for education and authorized testing purposes only.

Abstract

This project demonstrates the implementation of WAF (Web Application Firewall) to detect, monitor and mitigate malicious traffic on DVWA (Damn Vulnerable Web Application) server which is secured using SSL/TLS certificate. A Kali Linux machine was used to simulate common attacks like DDoS (Distributed Denial-Of -Service) and SQL Injection. This project evaluates the effectiveness and efficient mechanism of threat mitigation on Web Application Firewall based protection and elevated my view on, how vulnerable a web application can be without a rigid security measure.

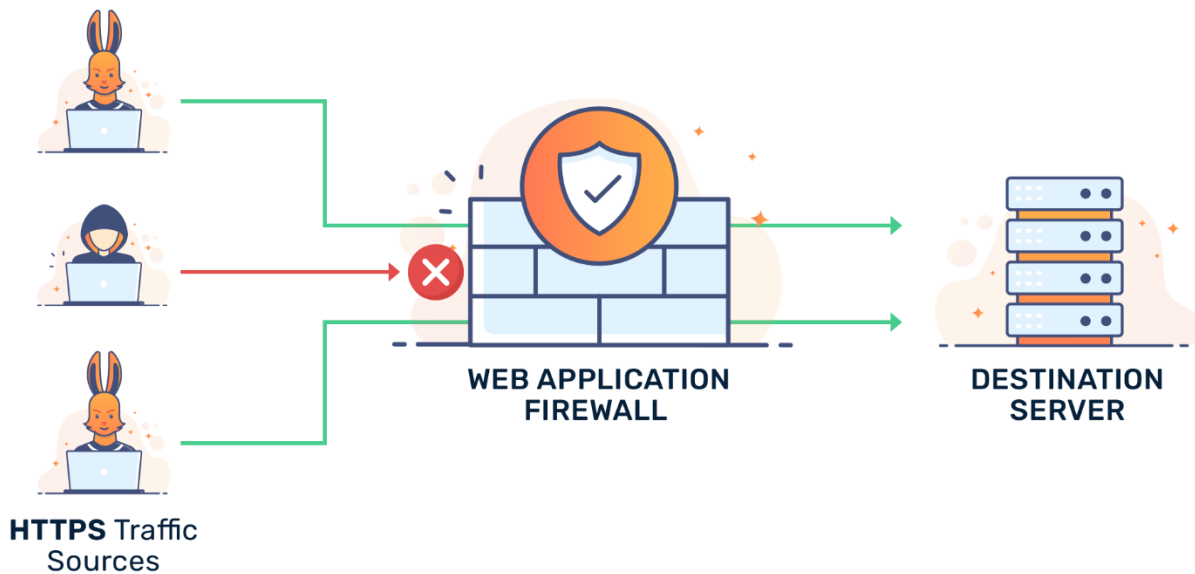
Table of Content

Introduction.....	5
Project Objectives.....	6
Environment Setup.....	7
1. DVWA Installation.....	7
1.1 Installing and configuring LAMP Stack.....	7
1.2 Securing MySQL server.....	8
1.3 Installing DVWA.....	8
1.4 Configuring DVWA Database.....	9
1.5 Configuring DVWA listening port to 8080.....	10
1.6 DNS Resolution Setup.....	10
1.7 Logging in to DVWA.....	11
2. Configuring SSL certificate.....	12
2.1 Creating a Certificate Signing Request.....	12
2.2 Creating a SSL Certificate.....	12
2.3 Testing through HTTPS request.....	13
3. WAF Installation.....	14
3.1 SafeLine WAF installation.....	14
3.2 Configuring SafeLine WAF.....	15
WAF Implementation.....	16
1. Authentication.....	16
2. Blacklisting a Malicious IP.....	17
3. DDoS (Distributed Denial of Service) Prevention.....	18
4. SQL Injection Prevention.....	20
Conclusion.....	22

Table of Figures

Figure 1: Installing Lamp Stack.....	7
Figure 2: Securing MySQL server.....	8
Figure 3: Installing DVWA.....	8
Figure 4: Making changes to DVWA config file.....	9
Figure 5: DVWA Default user credentials.....	9
Figure 6: Changing DVWA listening port to 8080.....	10
Figure 7: DNS resolution setup in Ubuntu.....	10
Figure 8: DNS resolution setup in Kali.....	11
Figure 9: DVWA home page.....	11
Figure 10: Creating a certificate signing request.....	12
Figure 11: Creating a SSL certificate.....	12
Figure 12: content of the Private key.....	13
Figure 13: Testing through https request.....	13
Figure 14: SafeLine WAF Installation (1).....	14
Figure 15: SafeLine WAF Installation (2).....	14
Figure 16: SafeLine WAF home page.....	15
Figure 17: Configuring SafeLine WAF.....	15
Figure 18: Enabling Authentication.....	16
Figure 19: Authentication in DVWA.....	16
Figure 20: Authentication Logs in WAF.....	17
Figure 21: Backlisted IP Logs in WAF.....	17
Figure 22: Access Forbidden to DVWA.....	18
Figure 23: Adding HTTP Flood Rule (1).....	18
Figure 24: Adding HTTP Flood Rule (2).....	19
Figure 25: DDoS Attack Successfully Prevented.....	19
Figure 26: HTTP Rate Limiting Logs.....	20
Figure 27: Performing a SQL Injection Attack.....	20
Figure 28: SQL Injection Successfully Prevented.....	21

Introduction



The web Application Firewall is a critical security measure that is used to protect web applications of organizations. WAF detects, filters and block malicious packets sent by threat actors. WAF detect and protects against common security flaws exploited in a web traffic and in this project I have used a self-hosted WAF. According to SANS institute web applications are the reason for 60% of the cyberattack attempts, that's because this is a way which requires the least effort but deals a big damage to an organization. SSL(secure socket layer) are equally important since it's a technology that encrypts the data sent between the website and the browser. Frequent cyberattacks that are used by the threat actors often are DDoS(process of overloading a website with purposeless requests obstructing the functionality of the real client) and SQL injection (here the attacker inserts a malicious code directly to the database hoping to bypass a user login or get user information without admin privileges).

Project Objective

- Deploy DVWA in an Ubuntu Desktop
- Configure HTTPS using SSL certificates
- Install and configure SafeLine WAF
- Simulate SQL Injection attacks
- Simulate DDoS attacks
- Monitor logs and traffic
- Evaluate WAF mitigation effectiveness

Environment Setup

1. DVWA Installation

1.1 Installing and configuring LAMP Stack

```
sudo apt-get install -y apache2 php php-mysql mysql-server
```

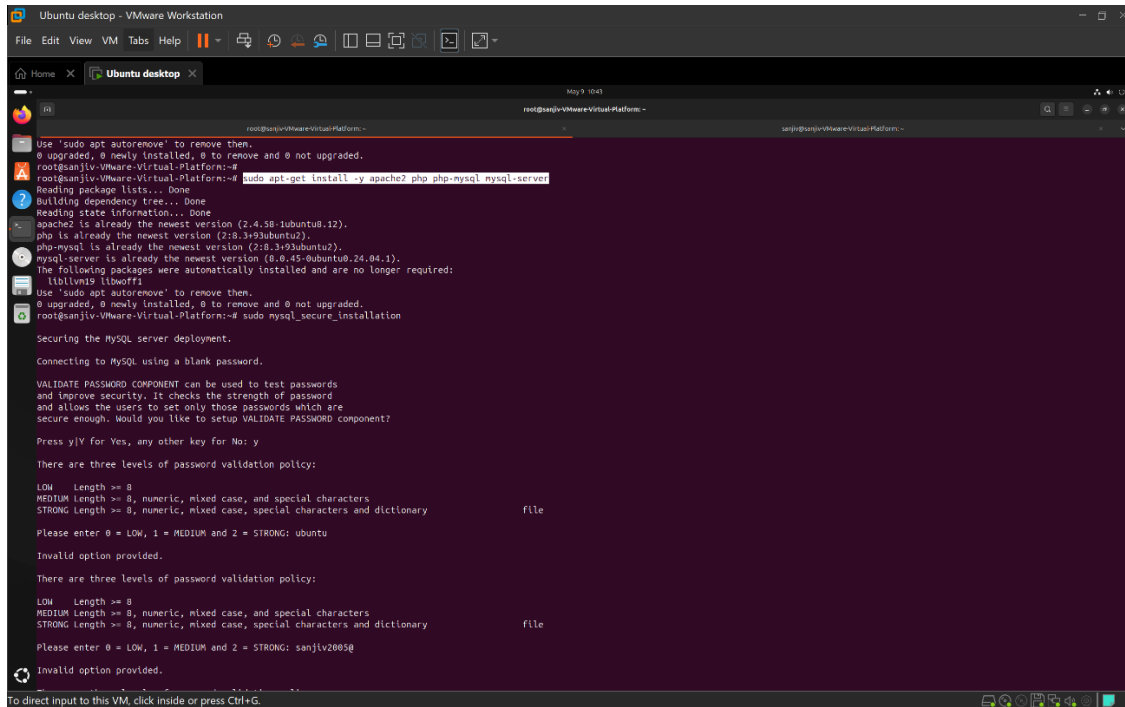


Figure 1: Installing Lamp Stack

This is a ubuntu package used to install all the core components needed for hosting a dynamic PHP web application with a backend database. This package includes apache2, which is responsible for listening for web requests and serving web pages to browsers, PHP the programming language that is responsible for dynamic web pages, database connected login systems, Php-mysql responsible for connecting the php application to the mysql database and MySQL server used for storing user accounts, passwords and company informations.

L – Linux

A – Apache

M – MySQL

P - PHP

1.2 Securing MySQL server

`sudo mysql_secure_installation`

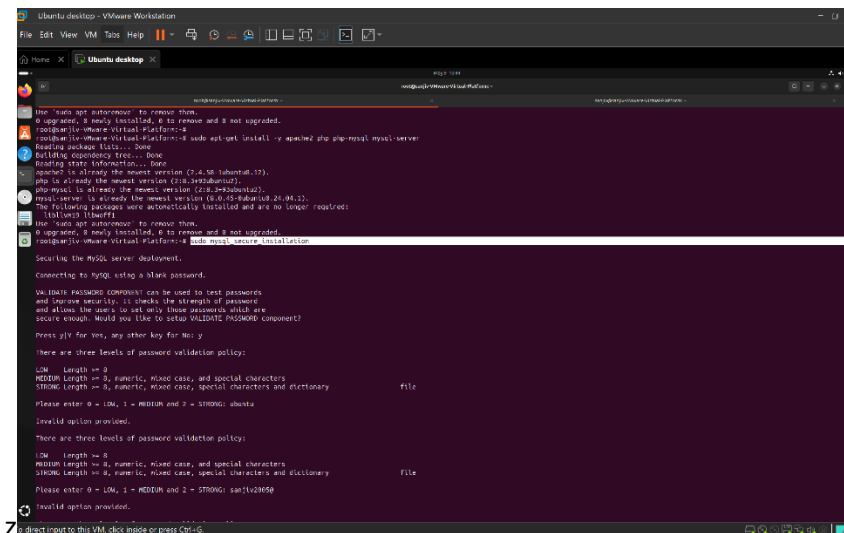


Figure 2: Securing MySQL server

This is used to secure the newly installed MySQL server, since it contains various security weaknesses, this is the best practice action taken by security professionals. By default SQL server allow insecure authentications, has anonymous users, allows remote root logins. This actions can act as a base level protection for SQL injection attacks.

1.3 Installing DVWA

`cd /var/www/html`

`sudo git clone https://github.com/digininja/DVWA.git`

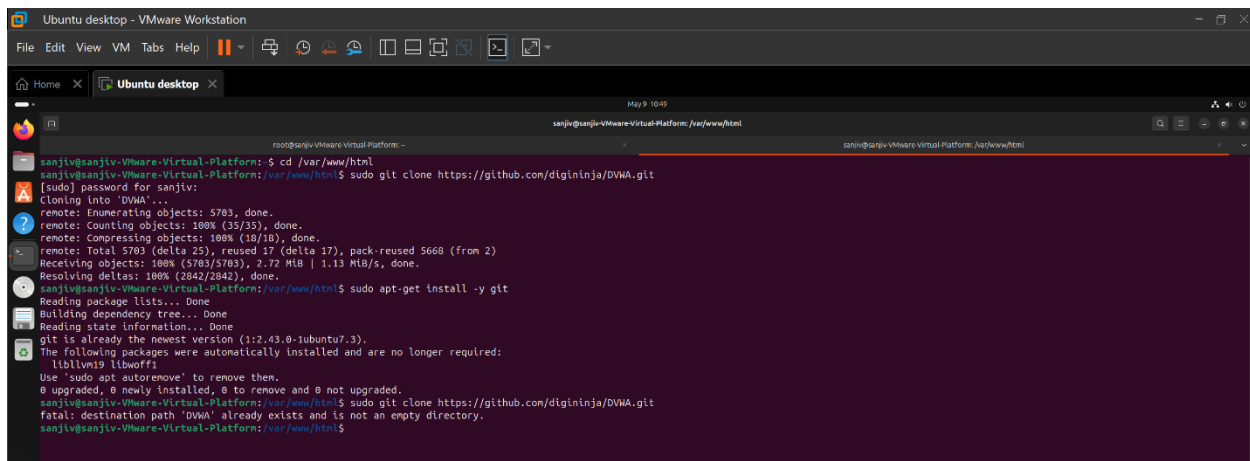


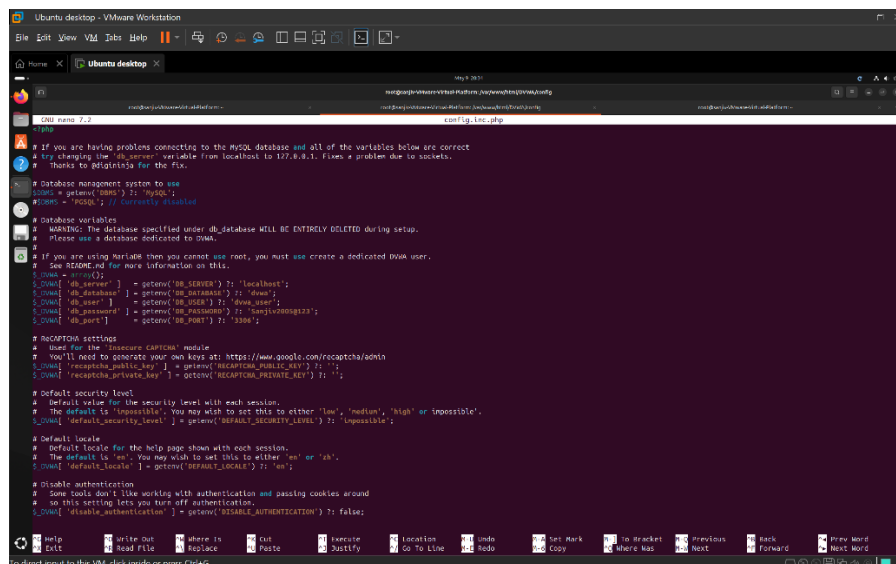
Figure 3: Installing DVWA

DVWA is a website that was deliberately made vulnerable for educational purposes. This comes with weak configurations. Here security levels can be adjusted from:

Low -> Medium -> High -> Impossible

1.4 Configuring DVWA Database

Vim DVWA/config/config.inc.php



```

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the $db_server variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
# Thanks to @p0p0r1a for the fix.

# Database management system to use
$dbms = getenv('DBMS') ?: 'mysql';
#DBMS = 'pgsql'; // Currently untested

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
# Please use a database dedicated to DVWA.
# If you are using XAMPP then you cannot use root, you must use create a dedicated dbwa user.
# See README.md for more information on this.
$rows = array();
$rows['db_server'] = getenv('DB_SERVER') ?: 'localhost';
$rows['db_database'] = getenv('DB_DATABASE') ?: 'dwa';
$rows['db_user'] = getenv('DB_USER') ?: 'dwa_user';
$rows['db_password'] = getenv('DB_PASSWORD') ?: 'Sanjiv2005@123';
$rows['db_port'] = getenv('DB_PORT') ?: '3306';

# reCAPTCHA settings
# Used for the 'Insecure CAPTCHA' module
# You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$rows['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$rows['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

# Default security level
# The default is 'low'. You may wish to set this to either 'low', 'medium', 'high' or 'impossible'.
$rows['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'low';

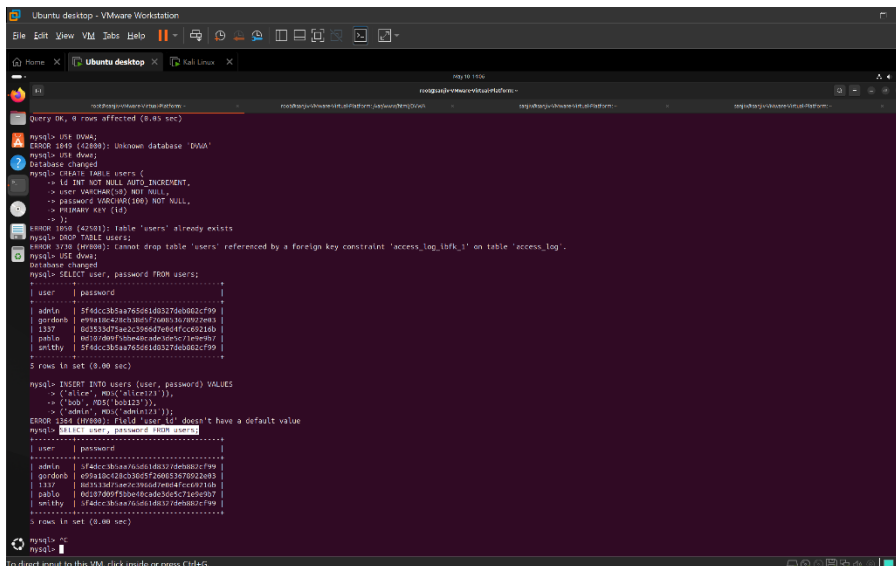
# Default locale
# Default locale for the help page shown with each session.
# The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$rows['default_locale'] = getenv('DEFAULT_LOCALE') ?: 'en';

# Disable authentication
# Some tools don't like working with authentication and passing cookies around
# So this setting lets you turn off authentication.
$rows['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ?: false;
  
```

Figure 4: Making changes to DVWA config file

In this config file, I had to edit DB_PASSWORD as “Sanjiv2005@123” because of the password strength requirement I established while securing the MySQL server.

`sudo mysql -u root -p`



```

mysql> show databases;
+-----+
| Database |
+-----+
| dwa      |
+-----+

mysql> use dwa;
Database changed

mysql> show tables;
+-----+
| tables_in_dwa |
+-----+
| users         |
+-----+

mysql> select * from users;
+----+-----+-----+-----+
| id  | user | password | access_log |
+----+-----+-----+-----+
| 1   | admin | $F6dc385a7651e803276d0602cf99 |  |
| 2   | gordonb | e9fa1dc12c030c5f2d06537052d083 |  |
| 3   | l33t | 00513079e0239e0d7e0d4fc0c921d0 |  |
| 4   | pablo | 00187009f3b0e0c0c5c71c0e907 |  |
| 5   | grrty | $F6dc385a7651e803276d0602cf99 |  |
+----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> insert into users (user, password) values
-> ('admin', MD5('admin123')),
-> ('bob', MD5('bob123'));
ERROR 1364 (30000): Field 'access_log' doesn't have a default value

mysql> select user, password from users;
+-----+-----+
| user | password |
+-----+-----+
| admin | $F6dc385a7651e803276d0602cf99 |
| gordonb | e9fa1dc12c030c5f2d06537052d083 |
| l33t | 00513079e0239e0d7e0d4fc0c921d0 |
| pablo | 00187009f3b0e0c0c5c71c0e907 |
| grrty | $F6dc385a7651e803276d0602cf99 |
+-----+-----+
5 rows in set (0.00 sec)

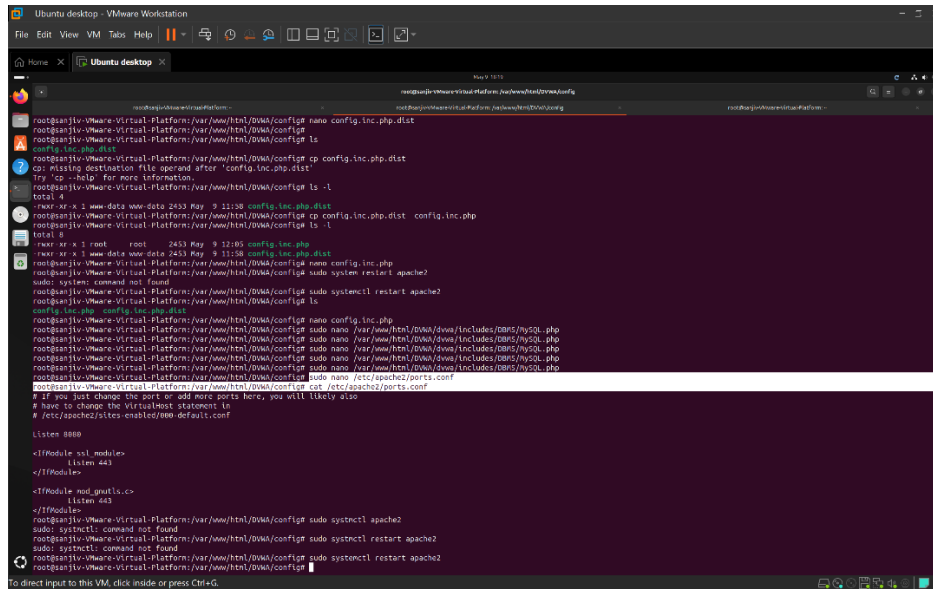
mysql>
  
```

Figure 5: DVWA Default user credentials

I proceeded with using the default database of DVWA web application instead of creating a custom database but here the passwords are in MD5 hash format, which I had to convert to plain text in order to access the web with the provided credentials.

1.5 Configuring DVWA listening port to 8080

sudo nano /etc/apache2/ports.conf



```

root@kali:~# nano /etc/apache2/ports.conf
root@kali:~# cat /etc/apache2/ports.conf
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/default.conf

Listen 8080

<IfModule mpm_module>
    Listen 443
</IfModule>

<IfModule mpm_module>
    Listen 443
</IfModule>

root@kali:~# systemctl restart apache2
root@kali:~# systemctl status apache2
root@kali:~# systemctl restart apache2

```

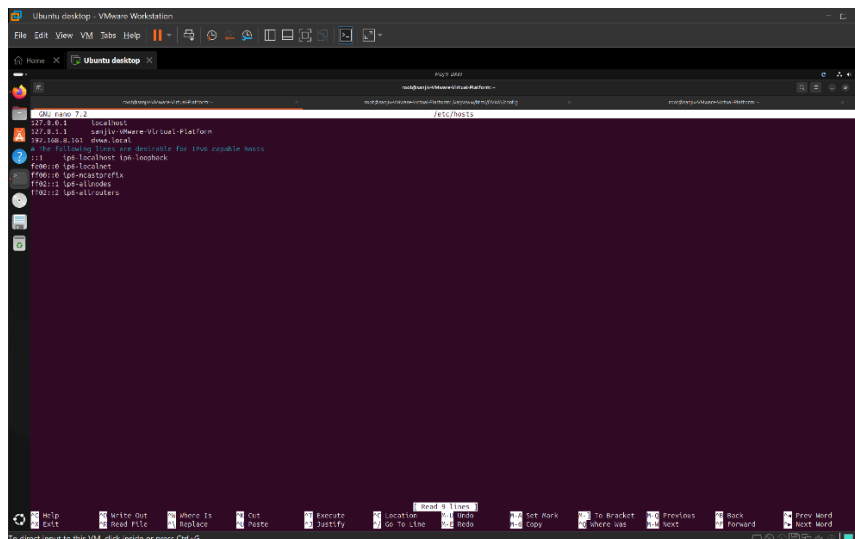
Figure 6: Changing DVWA listening port to 8080

Change the listening port of DVWA from 80 to 8080. 8080 is normally used in testing and development since it avoids conflicts between other web services and this also doesn't require root privileges.

1.6 DNS Resolution Setup

sudo nano /etc/hosts

Using the above command, I added a manual DNS resolution rule to override the DNS server and translate the DVWA ip address to a local hostname as dvwa.local. This set up was done to the DVWA server machine (Ubuntu) and attacker machine (Kali Linux).



```

root@kali:~# nano /etc/hosts
root@kali:~# cat /etc/hosts
127.0.0.1 localhost
127.0.0.1 dvwa.local
# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost
::1 ip6-localnet
::1 ip6-allnodes
::1 ip6-allrouters

```

Figure 7: DNS resolution setup in Ubuntu

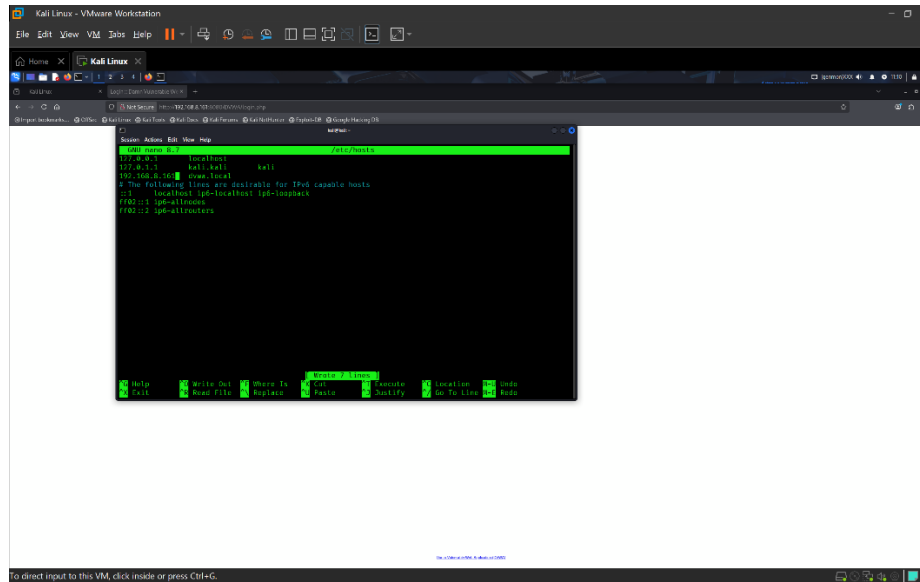


Figure 8: DNS resolution setup in Kali

Now I can access the webapp at <http://dvwa.local:8080/DVWA> in both my kali Linux (Attacker Machine) and Ubuntu (self-hosting the SafeLine WAF and the DVWA) browser.

1.7 Logging in to DVWA

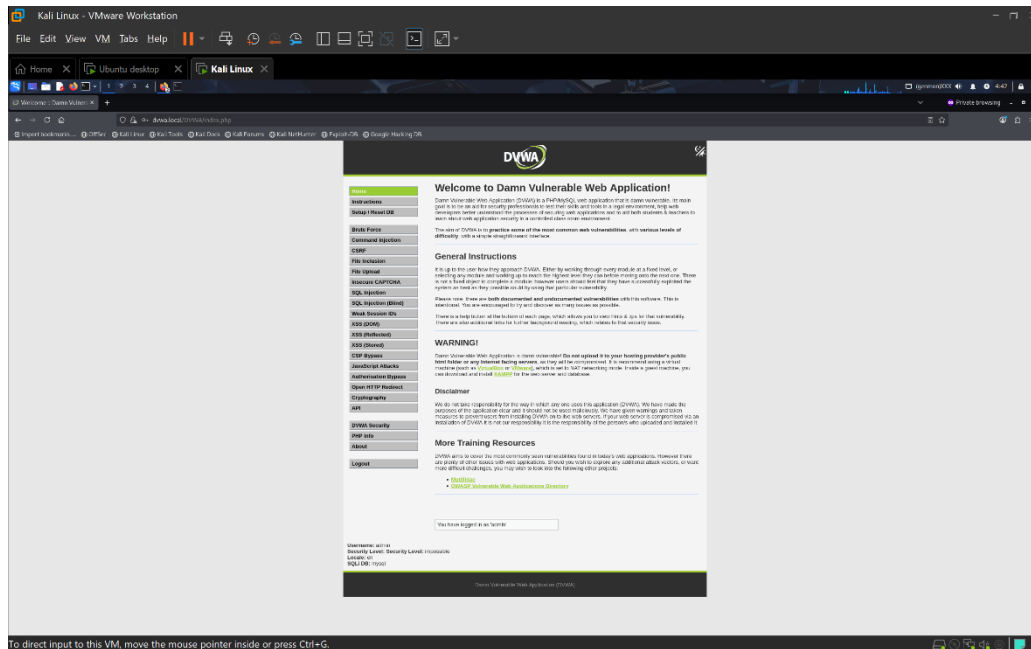


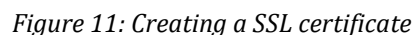
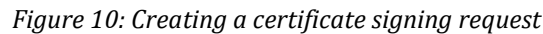
Figure 9: DVWA home page

Successfully created a fully functional DVWA, which contains various attack simulation facilities.

2.1 Creating a Certificate Signing Request

```
openssl req -new -key priv.key -out priv.csr -subj
```

This command is used to create a certificate signing request (csr) using the existing private key, I generated earlier. The CSR includes a organization details, domain name, digital signature of the private key.



openssl x509 -req -days 365 -in priv.csr -signkey priv.key -out priv.crt

This creates a self signed SSL certificate using the CSR and private key by reading the CSR, extracting the public key and signed it using my private key and then generates a certificate that is valid for 365 days as requested.

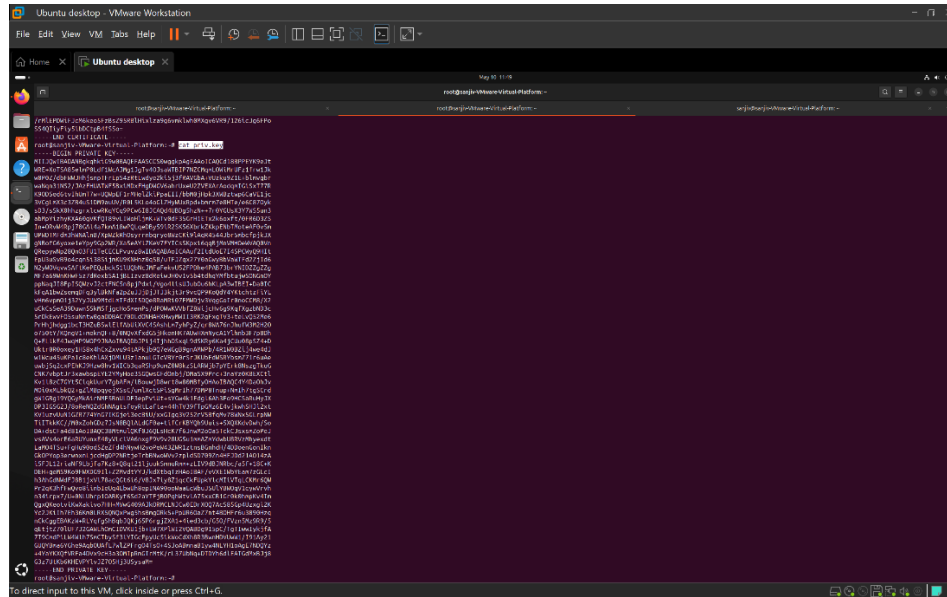


Figure 12: content of the Private key

2.3 Testing through HTTPS request

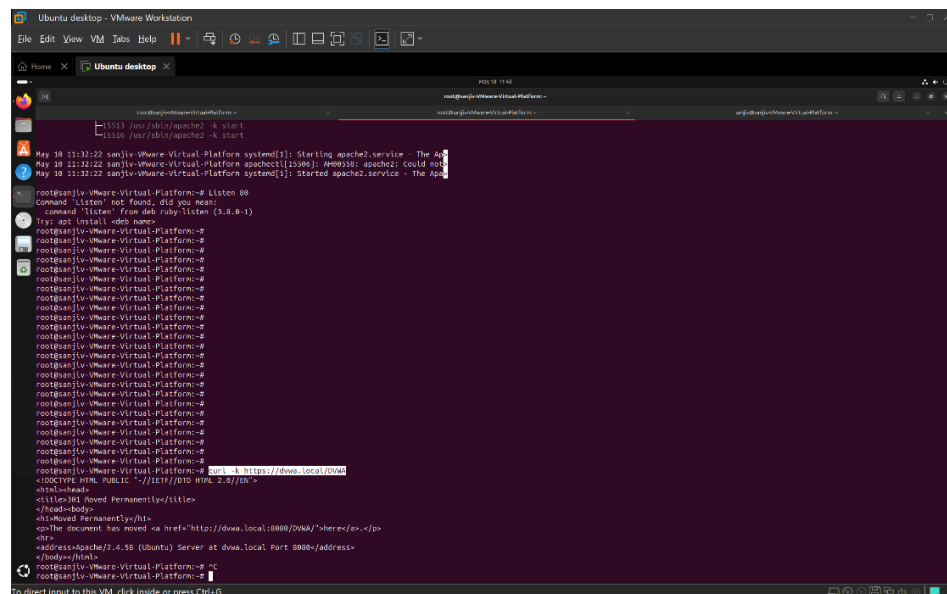


Figure 13: Testing through https request

curl -k https://dwa.local/DVWA

To check my setup I sent a https request bypassing the SSL certificate validation errors since curl would reject the request because, I used a self signed certificate instead of a authorized certificate

3.1 SafeLine WAF installation

Ubuntu desktop - VMware Workstation

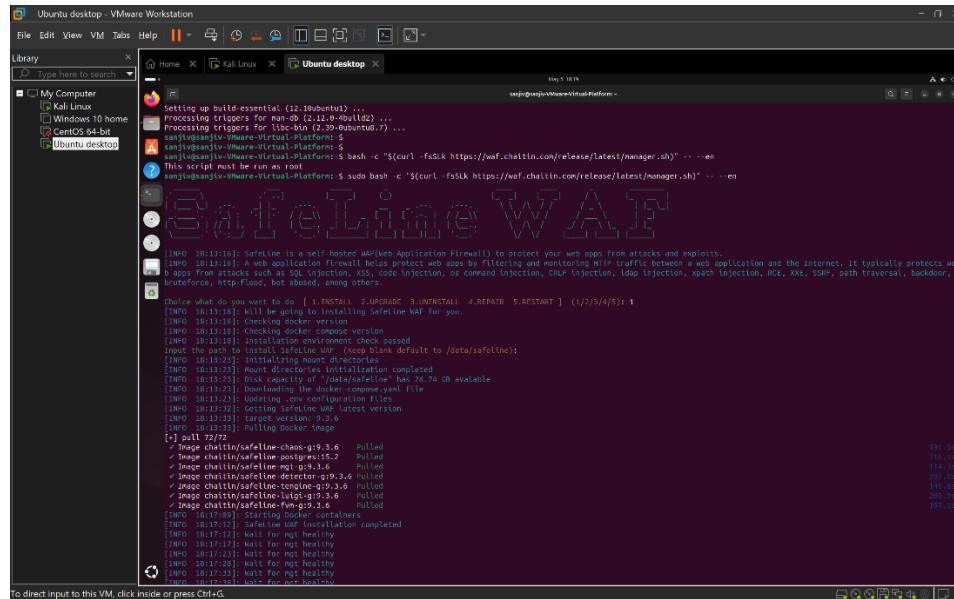


Figure 14: SafeLine WAF Installation (1)

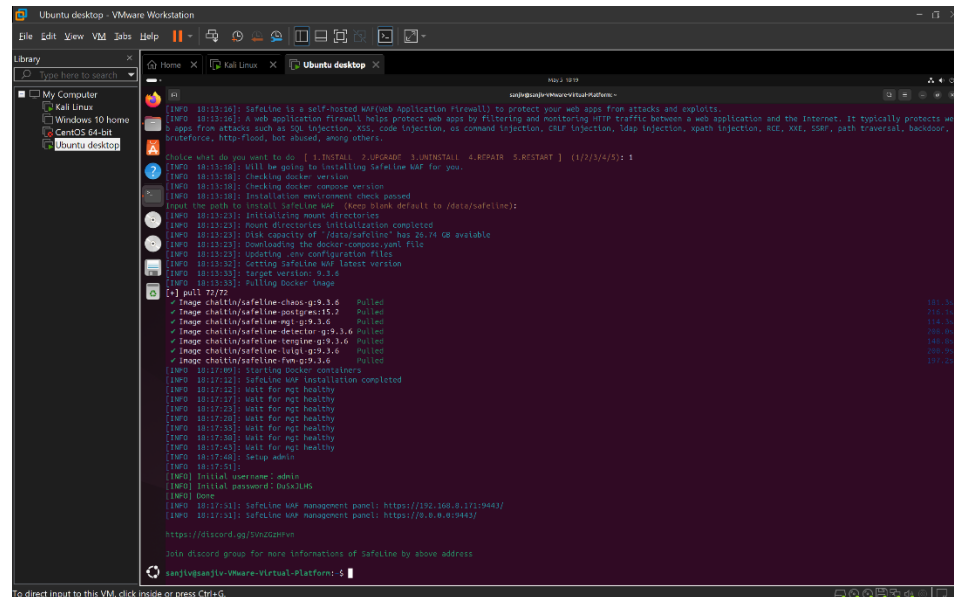


Figure 15: SafeLine WAF Installation (2)

Installed SafeLine web app and got the URL as <https://<Ubuntu IP>:9443/> and also a username and a password for login.

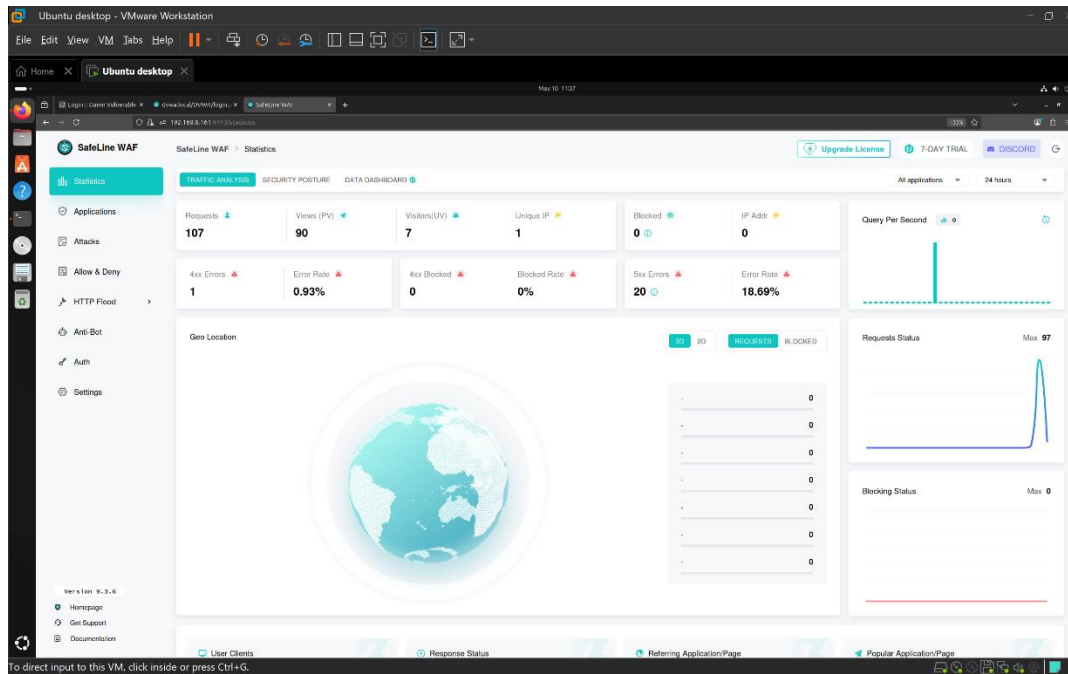


Figure 16: SafeLine WAF home page

3.2 Configuring SafeLine WAF

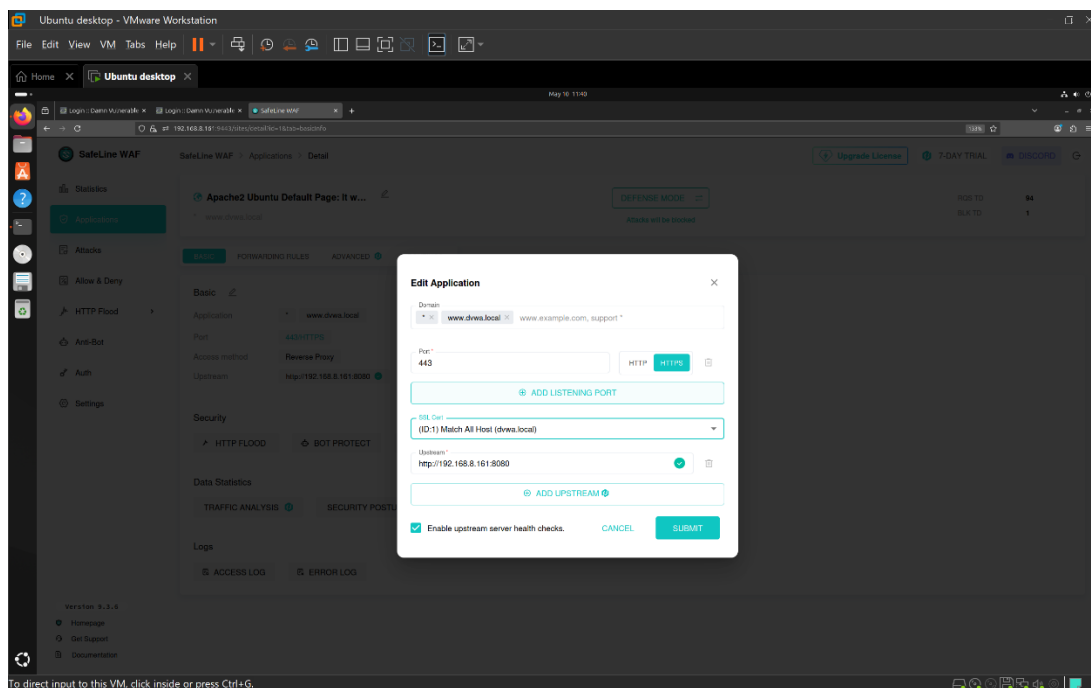


Figure 17: Configuring SafeLine WAF

Edited the web application with the Domain as **WWW.dvwa.local**, established the previously created **SSL Certificate** and upstreamed to the DVWA IP: **http://192.168.8.161:8080/**.

WAF Implementation

1. Authentication

SafeLine web application provides password authentication and TOTP (Time based One Time Password) authentication features to only allow authorized users to access the web content. I authenticated my kali Linux machine by assigning its IP address in the WAF.

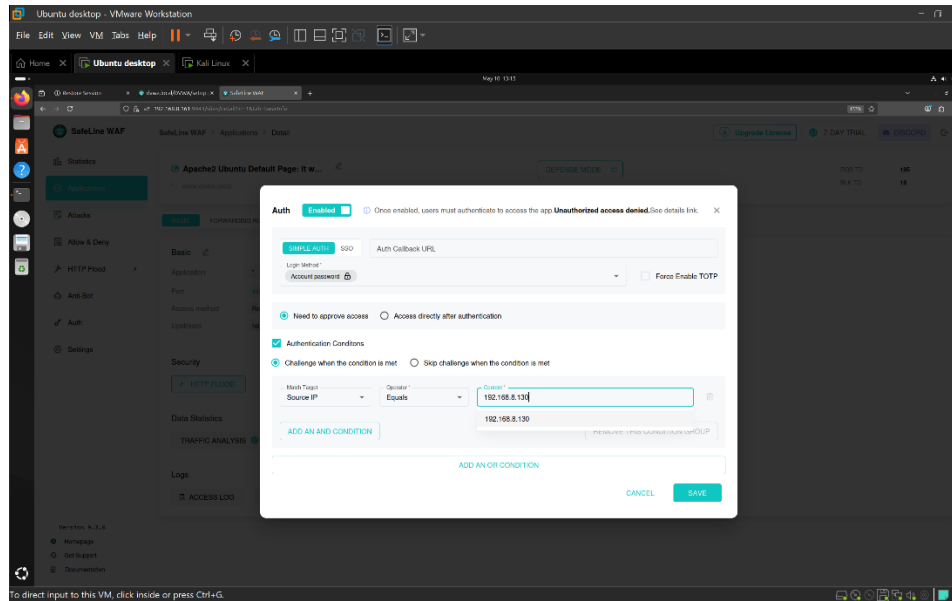


Figure 18: Enabling Authentication

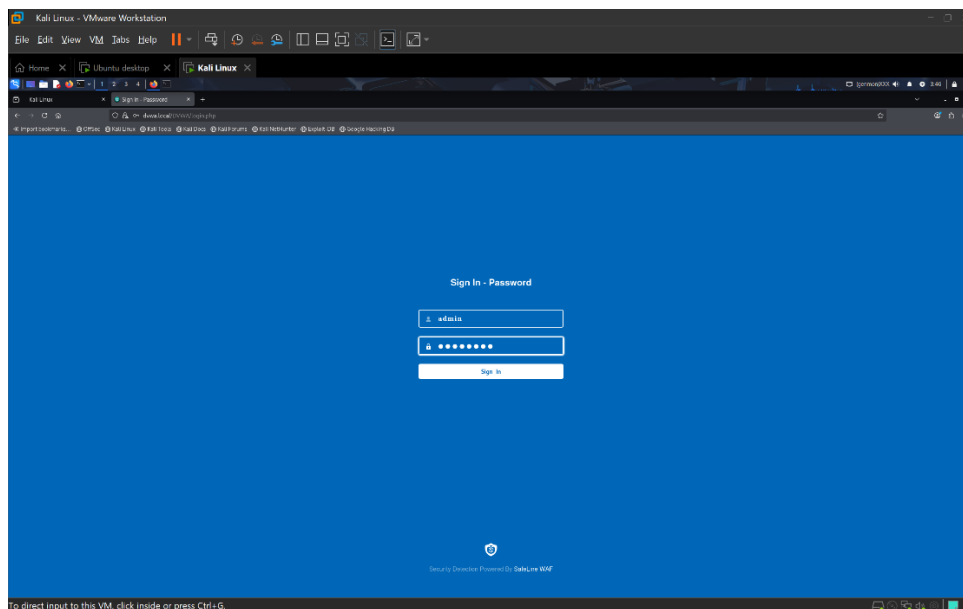


Figure 19: Authentication in DVWA

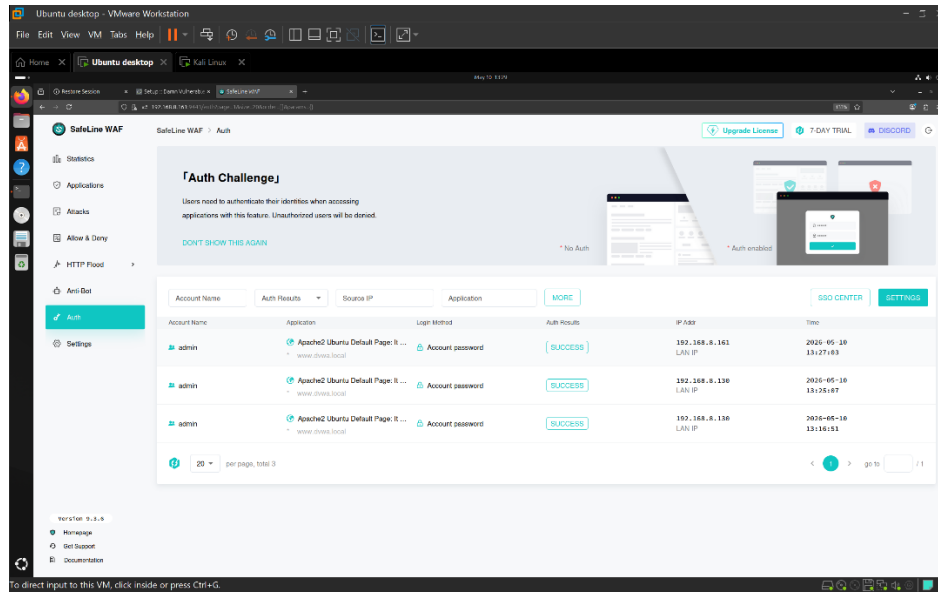


Figure 20: Authentication Logs in WAF

2. Blacklisting a Malicious IP

Blacklisting an IP address to completely shut down actions from the Specific malicious IP or a realized future threat machine.

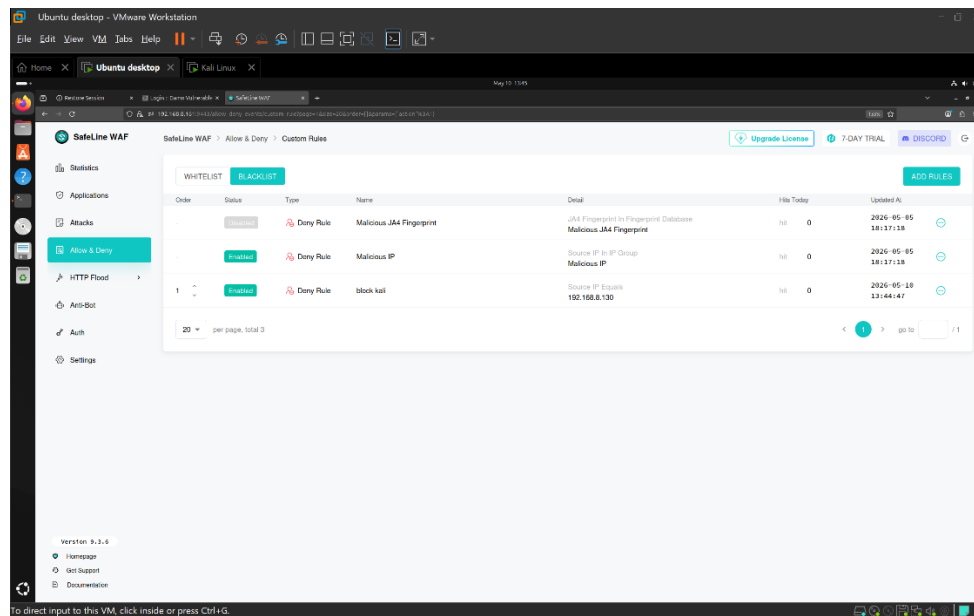


Figure 21: Blacklisted IP Logs in WAF

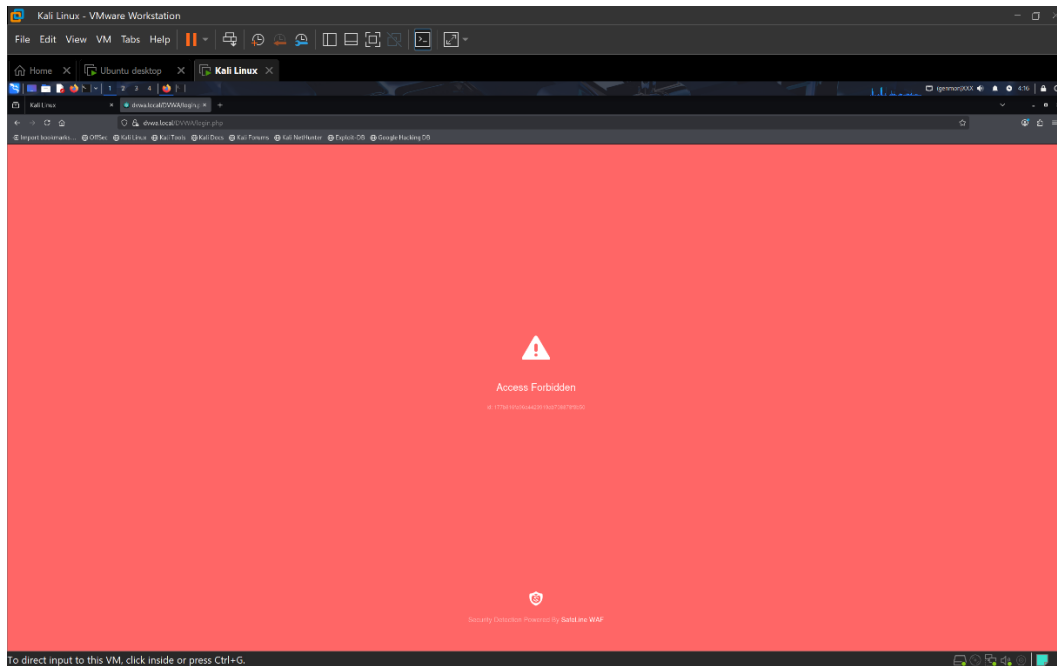


Figure 22: Access Forbidden to DVWA

3. DDoS (Distributed Denial of Service) Prevention

DDoS is an attack made to crash the website or a network by flooding it with internet traffic preventing legitimate users from accessing the website. Therefore, I Added a rule to block the user for **5 minutes** if he were to access the web **3 times** within **10 seconds**.

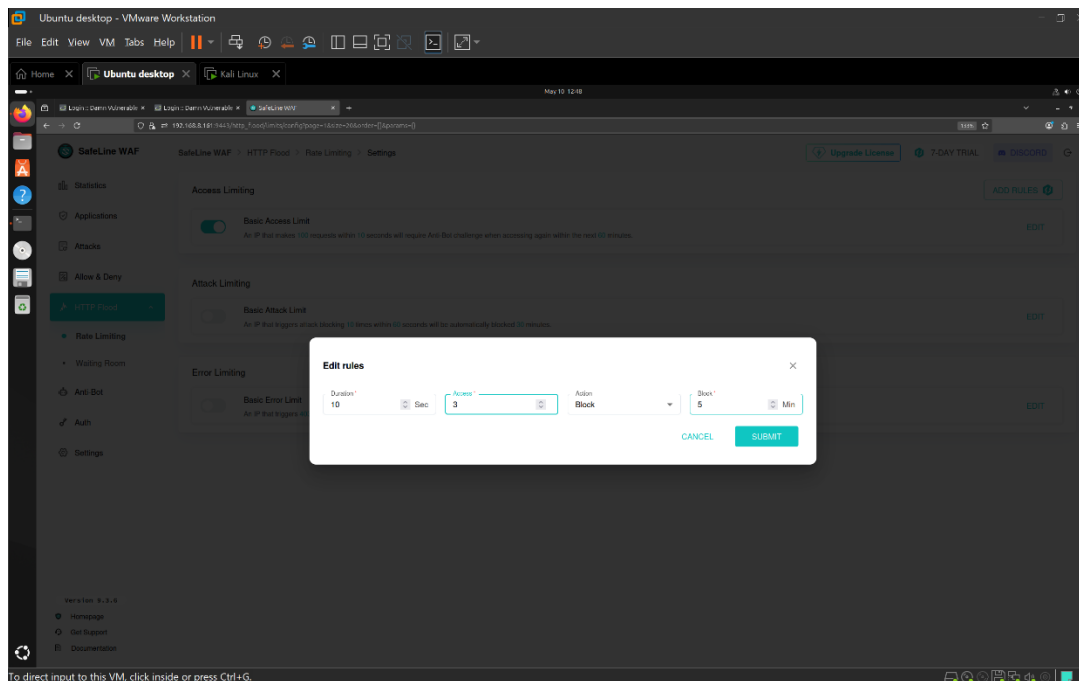


Figure 23: Adding HTTP Flood Rule (1)

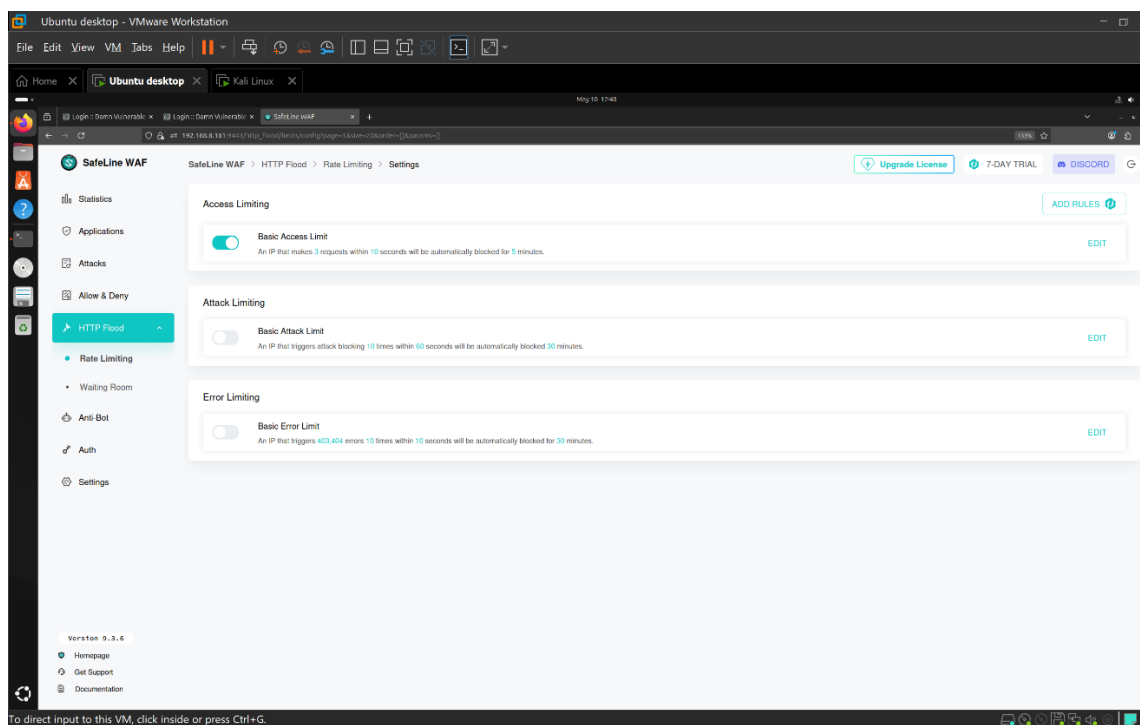


Figure 24: Adding HTTP Flood Rule (2)

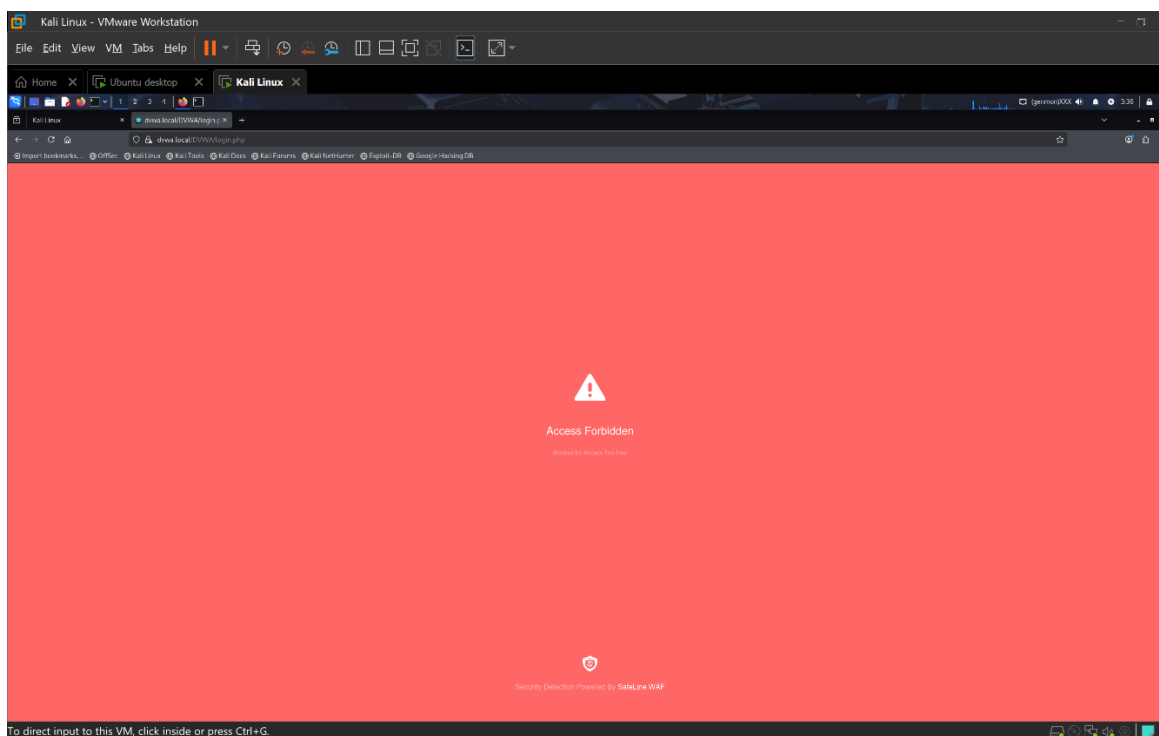


Figure 25: DDoS Attack Successfully Prevented

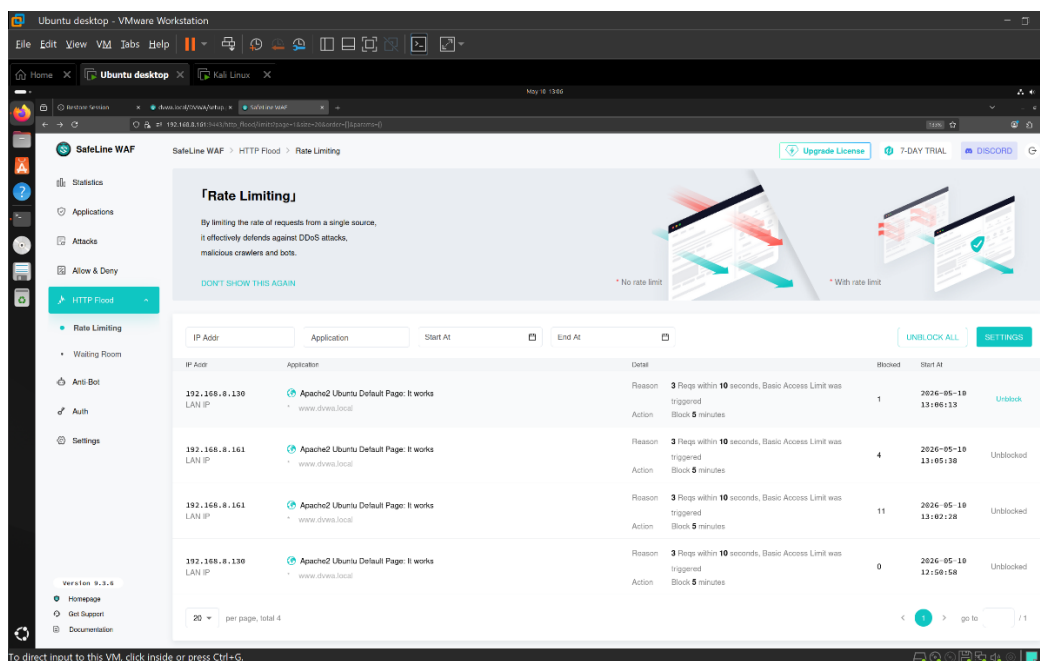


Figure 26: HTTP Rate Limiting Logs

4. SQL Injection Prevention

SQL Injection is when the threat actor inserts a malicious SQL code to manipulate the database query. This is the classic SQL code, that is used to bypass login or to return all database rows.

`'OR'1'='1`

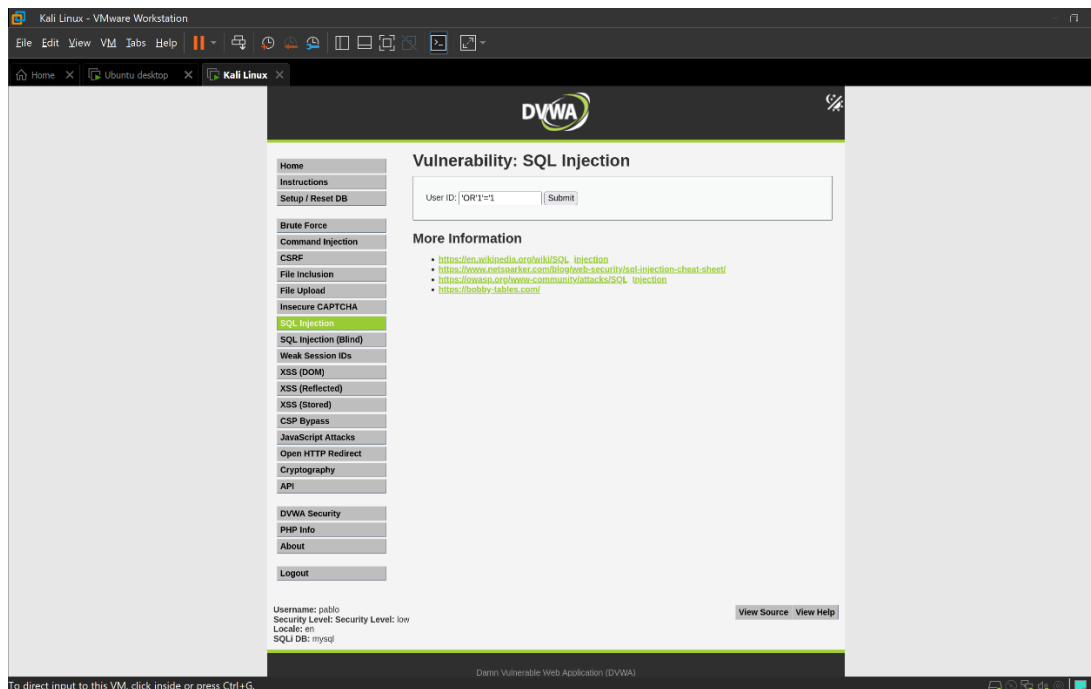


Figure 27: Performing a SQL Injection Attack

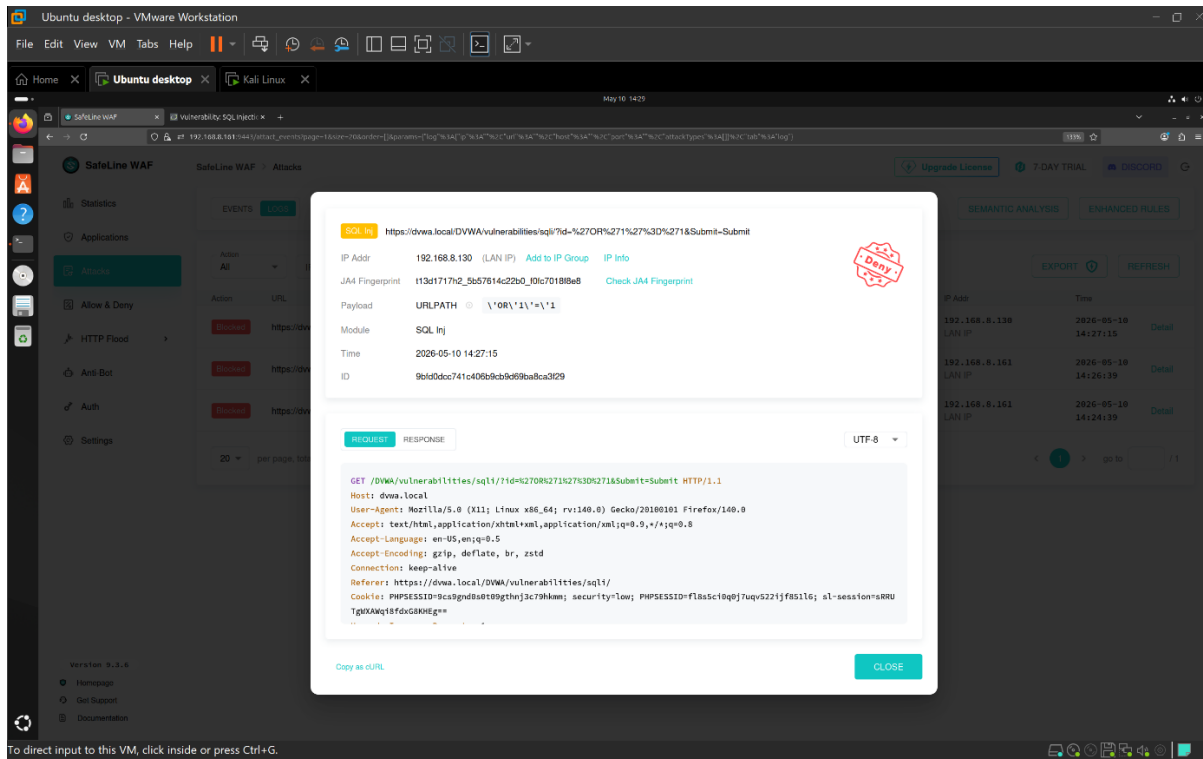


Figure 28: SQL Injection Successfully Prevented

Conclusion

This project successfully demonstrated the deployment, configuration, and evaluation of the SafeLine web application firewall. The objective of this project was to understand how a self-hosted web application firewall will detect, monitor and mitigate common web based attack. After deploying the SafeLine web application and a fully functioning DVWA secured by the SSL certificate, Multiple attack scenarios, including authentication testing, IP blacklisting, DDoS (Distributed Denial of Service) simulation, and SQL injection attacks were performed. The results show that SafeLine web application firewall was capable of identifying malicious traffic patterns, filtering suspicious requests and improving the overall security aspect of a web application.